Streamlit AutoML Agent + Gemini Chat

4-Day Team Execution Plan

MVP plan for a four-person team

Final MVP: Upload CSV → Select Target → Run AutoML → View Leaderboard + Plots → Chat with Gemini → Download Report/Model

Scope note: This plan compresses the original 9-phase AutoML proposal into a demo-ready MVP that can be completed in 4 days.

1. Project Vision

The project is a Streamlit-based AutoML Agent. The user uploads a tabular CSV dataset, selects the target column, and the system automatically preprocesses the data, trains several machine learning models, evaluates them, chooses the best model, explains the result, and lets the user chat with Gemini about the experiment.

Simple explanation: the app acts like a mini data scientist inside a website. Streamlit is the face, the AutoML backend is the worker, and Gemini is the explainer.

2. MVP Features

Must-Have Feature	Description
CSV upload	User uploads a dataset from the Streamlit interface.
Dataset preview	Show first rows, number of rows/columns, missing values, and basic summary.
Target selection	User selects the column to predict.
Task detection	Auto-detect classification or regression, with manual override.
Preprocessing	Handle missing values, scale numeric features, and encode categorical features.
Model training	Train 3 core models for classification or regression.
Leaderboard	Rank models by weighted F1 for classification or RMSE for regression.
Best model	Show the winning model and save it as a joblib file.
Explainability	Show feature importance and confusion matrix or prediction-vs-actual plot.
Gemini chat	Let the user ask questions about the actual experiment results.
Report/downloads	Export leaderboard, report, plots, and saved model.

3. Features to Avoid Until the MVP Works

- ☐ Full SHAP explanations for every model.
- ☐ Full Optuna tuning for all models.
- ☐ Stacking ensemble and complex blending.
- ☐ Excel/JSON upload support.
- ☐ User login, database, or authentication.
- ☐ Advanced CSS/animations.
- ☐ Large dataset optimization.
- ☐ Full multi-iteration automatic agent loop.

4. Team Roles

Member	Role	Main Files	Core Responsibility
Member 1	Streamlit UI Owner	app.py, ui_components.py	Build the interface, tabs, upload flow, result display, chat UI, and download buttons.
Member 2	AutoML Backend Owner	automl/preprocessor.py, model_registry.py, trainer.py	Build preprocessing, model registry, training logic, task detection, and model saving.
Member 3	Evaluation, Plots, Report Owner	automl/evaluator.py, explainer.py, reporter.py	Build metrics, leaderboard, plots, feature importance, and report generation.
Member 4	Gemini + Integration	llm/gemini_client.py,	Connect Gemini, build prompts, merge

	Lead	prompts.py, automl/orchestrator.py	components, manage result object, and prepare final integration.

5. Recommended Folder Structure

```
automl-agent/
|
├── app.py
├── requirements.txt
├── README.md
├── .env
|
├── automl/
|   ├── __init__.py
|   ├── preprocessor.py
|   ├── model_registry.py
|   ├── trainer.py
|   ├── evaluator.py
|   ├── explainer.py
|   ├── reporter.py
|   └── orchestrator.py
|
├── llm/
|   ├── __init__.py
|   ├── gemini_client.py
|   └── prompts.py
|
├── data/
|   ├── heart.csv
|   └── housing.csv
|
└── outputs/
    ├── plots/
    ├── leaderboard.csv
    ├── report.md
    └── best_model.joblib
```

6. Day 1 — Working Skeleton

Main goal: CSV upload → target selection → train 2 basic models → show leaderboard.

Owner	Tasks	Deliverable
Member 1 — UI	Create app.py; add title; add CSV uploader; read CSV with pandas; show preview; add target dropdown; add task selector; add Run AutoML button; create tabs.	app.py can upload and preview a dataset.
Member 2 — Backend	Create preprocessing; detect numeric/categorical columns; impute missing values; scale numeric columns; one-hot encode categorical columns; create train/test split; add Logistic Regression, Random Forest, Linear Regression, Random Forest Regressor.	Backend trains basic models.
Member 3 — Evaluation	Create evaluator.py; calculate accuracy and weighted F1 for classification; calculate RMSE, MAE, and R2 for regression; return sorted leaderboard.	Clean leaderboard DataFrame.
Member 4 —	Create orchestrator.py; implement run_automl(df, target_col,	Clicking Run AutoML shows

Integration	task_type); connect Streamlit Run button to backend; store results in session_state.	leaderboard.

Day 1 success condition: streamlit run app.py → upload CSV → select target → click Run AutoML → see leaderboard.

7. Day 2 — Better Models, Results, and Visuals

Main goal: The app shows dataset summary, better leaderboard, best model card, and at least one useful plot.

Owner	Tasks	Deliverable
Member 1 — UI	Improve layout with tabs and columns; add progress spinner; add dataset summary cards; show best model card; add clear error messages.	App looks organized and presentable.
Member 2 — Backend	Add Gradient Boosting models; optionally add SVM/SVR; add simple hyperparameters for Random Forest; keep training fast.	Backend trains 3-4 models per task.
Member 3 — Plots	Add confusion matrix for classification; prediction-vs-actual plot for regression; feature importance using tree importance or permutation importance; save plots to outputs/plots.	Explainability tab shows useful plots.
Member 4 — Integration	Create one stable result object containing dataset summary, task type, leaderboard, best model name, metrics, feature importance, plot paths, and model path; save leaderboard and best model.	All results are available for Streamlit, Gemini, and report.

Day 2 success condition: Upload tab, Results tab, and Explainability tab all work.

8. Day 3 — Gemini Chat and Report

Main goal: The user can chat with Gemini about the actual AutoML results and download a report.

Owner	Tasks	Deliverable
Member 1 — Chat UI	Add Gemini Chat tab; use st.chat_message and st.chat_input; store chat history; show suggested questions.	Chat interface appears and keeps history.
Member 2 — Stability	Test one classification dataset and one regression dataset; fix preprocessing, categorical, missing value, and model errors; add clear exceptions.	Pipeline works on both classification and regression datasets.
Member 3 — Report	Create reporter.py; generate report.md; include summary, task type, leaderboard, best model, metrics, feature importance, Gemini analysis, and suggested improvements.	outputs/report.md is generated and downloadable.
Member 4 — Gemini API	Create gemini_client.py; read GEMINI_API_KEY from .env or Streamlit secrets; create ask_gemini(question, results); build strict prompt; return Gemini answer; add fallback on API failure.	Gemini answers using actual experiment results.

Day 3 success condition: Run AutoML → open Gemini Chat → ask "Why did this model win?" → get an answer based on leaderboard and feature importance → download report.md.

9. Gemini Prompt Template

```
You are an ML assistant inside an AutoML application.
```

```
You must answer only using the experiment results below.
Do not invent scores, models, or features.
If something is not available, say that it is not available.
Explain in simple language and suggest practical improvements.

Dataset summary:
{dataset_summary}

Task type:
{task_type}

Leaderboard:
{leaderboard}

Best model:
{best_model_name}

Metrics:
{metrics}

Feature importance:
{feature_importance}

User question:
{user_question}
```

10. Day 4 — Polish, Deployment, and Demo

Main goal: No big new features. Only fixing, cleaning, testing, deployment attempt, and demo preparation.

Owner	Tasks	Deliverable
Member 1 — UI Polish	Improve title and description; add sidebar instructions; add warning messages; add loading messages; make download buttons clear; hide empty sections until results exist.	App is easy to understand and demo-ready.
Member 2 — ML Testing	Test classification and regression end-to-end; limit training time; remove models that crash; ensure best_model.joblib is saved and reloadable.	Backend is stable.
Member 3 — Final Outputs	Finalize leaderboard.csv, report.md, and plots; add screenshots for README; prepare demo dataset; prepare short metric explanations.	All final files are generated correctly.
Member 4 — Deployment + README	Final integration check; create requirements.txt and README.md; add setup instructions and Gemini API key instructions; attempt Streamlit Community Cloud deployment; prepare demo script.	Project can be run by another person.

Day 4 success condition: A demo-ready Streamlit app with working upload, AutoML run, leaderboard, plots, Gemini chat, report download, and saved model.

11. Daily Time Blocks

Day	Time Block	Focus
Day 1	Hour 1	Create GitHub repo, folder structure, branches, and assign files.
Day 1	Hours 2-4	Build UI upload page, preprocessing, basic models, and basic metrics.
Day 1	Hours 5-7	Connect UI to backend and test first dataset.

Day 1	Hour 8	Fix errors and commit working version.
Day 2	Hours 1-2	Clean bugs from Day 1.
Day 2	Hours 3-5	Add more models, plots, and best model card.
Day 2	Hours 6-7	Save result object, model, and leaderboard.
Day 2	Hour 8	Full app test.
Day 3	Hours 1-2	Add Gemini API client and chat UI.
Day 3	Hours 3-4	Build prompt using real results.
Day 3	Hours 5-6	Generate report.md.
Day 3	Hours 7-8	Test classification/regression and fix integration bugs.
Day 4	Hours 1-2	UI polish and error handling.
Day 4	Hours 3-4	Final testing.
Day 4	Hour 5	README and requirements.
Day 4	Hour 6	Deployment attempt.
Day 4	Hours 7-8	Demo script and final rehearsal.

12. Recommended Models and Scoring Rules

Task Type	Models	Main Score	Best Model Rule
Classification	Logistic Regression, Random Forest Classifier, Gradient Boosting Classifier. Optional: SVM, KNN.	Weighted F1-score	Highest weighted F1-score.
Regression	Linear Regression, Random Forest Regressor, Gradient Boosting Regressor. Optional: SVR, KNN Regressor.	RMSE	Lowest RMSE.

Important: Use scikit-learn models first. Avoid XGBoost unless the environment is already stable. A working demo with 3 models is better than a broken app with 7 models.

13. GitHub Workflow

```
Branches:
main
ui-streamlit
backend-automl
evaluation-report
gemini-integration

Daily workflow:
1. Work on your branch.
2. Push changes.
3. Open pull request.
4. Integration lead merges carefully.
5. Test main branch before ending the day.
```

14. Final MVP Checklist

- ☐ CSV upload works.
- ☐ Dataset preview appears.
- ☐ Target column selection works.
- ☐ Classification/regression detection works.

- ☐ Preprocessing handles numeric, categorical, and missing values.
- ☐ At least 3 models train successfully.
- ☐ Leaderboard is shown and sorted correctly.
- ☐ Best model is selected and displayed.
- ☐ Feature importance is shown.
- ☐ Confusion matrix or regression plot is shown.
- ☐ Gemini chat answers based on actual results.
- ☐ Report is downloadable.
- ☐ Best model is saved as joblib.
- ☐ README explains setup and API key usage.
- ☐ Demo dataset is included.

15. Final Presentation Flow

- ☐ Open the Streamlit app.
- ☐ Upload the demo classification dataset.
- ☐ Select the target column.
- ☐ Run AutoML.
- ☐ Show dataset summary.
- ☐ Show leaderboard and best model.
- ☐ Show feature importance and confusion matrix.
- ☐ Ask Gemini: “Why did this model win?”
- ☐ Show the report download.
- ☐ Explain future work: full SHAP, Optuna, ensembling, and automatic rerun loop.

16. Team Presentation Lines

Member	Suggested Line
Member 1	I worked on the Streamlit interface: file upload, dataset preview, target selection, result tabs, chat UI, and download buttons.
Member 2	I worked on the AutoML backend: preprocessing, task detection, model registry, training, and saving the best model.
Member 3	I worked on evaluation and explainability: metrics, leaderboard, confusion matrix, regression plots, feature importance, and report generation.
Member 4	I worked on the Gemini agent and integration: connecting the ML results to Gemini so users can ask questions about the trained models.

17. Final Advice

Build in this order: working app → working ML pipeline → leaderboard → plots → Gemini chat → report → polish.

Do not start with: perfect tuning, full SHAP, ensembling, or beautiful CSS. A simple working demo will beat an advanced broken project.